

Claudemaxxing

15 Hidden Claude Code Features Most People Never Use

by **Mark Kashef**

Early AI Dopters

What's inside

Boris Cherny, the creator of Claude Code, shared his 15 favorite hidden and underutilized features in a thread that hit over 2 million views in 24 hours. This guide breaks down every single one with the practical details, exact syntax, pro tips, and the analogies that make each feature click. From voice coding to parallel worktrees to autopilot mode, these are the features that separate beginners from power users.

- 01 Mobile App
- 02 Teleport
- 03 Remote Control
- 04 Loop and Schedule
- 05 Hooks
- 06 Cowork Dispatch
- 07 Chrome Extension
- 08 Live Preview
- 09 Session Forking
- 10 /btw Side Queries
- 11 Git Worktrees
- 12 Batch Command
- 13 The --bare Flag
- 14 The --add-dir Flag
- 15 Custom Agents

Bonus: Voice Input

Cheat Sheet

01

Mobile App

Code from your phone. For real.

Claude Code has a full mobile app for iOS. Boris says he writes a lot of his code from his phone. It's not a watered-down version. You can start a session, write prompts, review code, and even commit changes without opening your laptop.

They're constantly working toward feature parity with the desktop version, so eventually you'll be able to run the majority of Claude Code from your phone's terminal.

Why mobile over other tools?

If you're using something like OpenClaw or a derivative, you can't always see what's happening. The Claude Code app shows you the bash executions, the actual code changes, all the transparency you'd get on desktop. For code-intensive tasks, that visibility matters.

The GitHub workflow:

- Create a GitHub repo that both your phone and desktop log into
- Push code from your phone, pull it on your desktop (or vice versa)
- Use **Plan Mode** on mobile (tap the code tab, bottom left) to do all your planning on the go
- Push the plan to your GitHub repo, then pull it from a desktop session and continue executing

Pro tip

Plan Mode on mobile is underrated. Do all your thinking and planning during your commute, push it, then execute from your desk when you get home.

Teleport

Resume a phone session on your desktop instantly.

Teleport lets you take an unsaved session from your mobile app and pick it up on your desktop computer. No copy-pasting, no screenshots, no starting over. The full conversation and context travels with you.

How it works:

- Start a session on your phone (e.g., 'read this website and update the hero section')
- When you get home, open your terminal
- Type the command below
- It shows your recent cloud sessions with timestamps
- Select the one you want and you're right back where you left off

```
claude --teleport
```

Pro tip

You don't need to do anything special from the app itself. Just open your terminal and run the command. It will find your cloud sessions automatically.

Remote Control

Control your desktop session from your phone.

Remote Control is the reverse of Teleport. Instead of moving a mobile session to desktop, you take a desktop session and control it from your phone. The session stays on your machine, you just steer it remotely.

This is perfect for monitoring heavy tasks. You kicked off something big on your laptop and want to watch the logs, see the progress, and send follow-up prompts while you're away from your desk.

```
/remote-control
```

Name your sessions:

When you have 5-6 remote control sessions running, they can get confusing. You can name each one so you always know which is which:

```
/remote-control "website redesign"
```

Now when you pull up your phone, that session shows up with the name you gave it instead of a random first prompt.

When to use what

Teleport = move the session to a new device (session moves)

Remote Control = steer an existing session from another device (session stays)

Dispatch = full computer control (mouse, keyboard, screen)

04

Loop and Schedule

Put Claude Code on autopilot.

This is how Boris and his team run Claude Code on autopilot. Two features, one core idea: execute a task on a repeating interval.

/loop runs as long as your terminal session and your computer are on. When you close the terminal, it stops.

/schedule runs in the cloud. It survives session restarts and can run for up to a week.

```
/loop 30m check my email inbox. Draft professional responses for anything that needs a reply. Save drafts but don't send anything.
```

When you execute this, Claude Code creates something called a cron job. You'll see:

- The **job ID** (to cancel it later)
- The **interval** (how often it runs)
- The **task** (what it executes)
- The **auto-expiry** (it stops automatically unless you cancel sooner)

Boris's use cases:

- Babysitting PRs every 5-10 minutes
- Pulling Slack feedback every 30 minutes
- Checking build status on a loop

Key difference

/loop needs your terminal open. /schedule runs in the cloud and survives restarts. Use /loop for quick monitoring, /schedule for longer automation.

Hooks

Rules that automatically fire based on events.

The analogy: Think of your iPhone. Before you go to bed, it automatically goes into Do Not Disturb mode. That's a hook. It fires based on an event (your bedtime) and executes an action (silence notifications). When you wake up and start using your phone, it asks 'do you want to turn off Do Not Disturb?' That's another hook.

Hooks in Claude Code work exactly the same way. You attach an action to an event, and it fires every time.

Available hook points:

- **SessionStart** - Fires when you open Claude Code. Auto-load context, check git status, surface pending tasks.
- **PreToolUse** - Fires before Claude uses any tool. Add validation, logging, or safety checks.
- **PostToolUse** - Fires after a tool runs. Log results, trigger notifications.
- **AfterEdit** - Fires after any file edit. Run linting, formatting, or tests automatically.
- **SessionEnd** - Fires when a session ends. Clean up temp files, log what happened.

Example: Auto-inject brand guidelines every session

```
Set up a hook so that every time I start a new session in this project,  
you automatically read the brand guidelines
```

Hooks vs CLAUDE.md: The critical difference

CLAUDE.md is a suggestion. Claude Code can choose to ignore it. A hook is deterministic. It WILL fire and it WILL execute, every single time. The content gets shoved into the context window whether Claude Code wants it or not.

Syntax help

A hook is just code in a settings.json file. If you don't want to write it manually, tag the **@claude-code-guide** agent in your prompt and it will write the hook with the correct syntax for you.

Cowork Dispatch

Give Claude Code full control of your desktop.

Dispatch is essentially remote control for your entire computer. Claude Code can use your MCPs, your browser, click on things, type, navigate applications, all with your permission.

Setup:

- Download the Claude Code desktop app
- Navigate to Dispatch in the settings
- Whitelist the access requests (what Claude can and can't control)
- Toggle on **Keep Awake** (prevents your Mac from sleeping mid-task)
- Toggle on **Enable Computer Use**

When you'd actually use this:

A practical example: if you have a Chrome extension and you want Claude Code to physically click on it, reload it, and test it, regular tools can't do that. They can access things through the browser's network tab, but they can't physically click on a UI element. Dispatch can. It takes full control and interacts with your computer the way you would.

Pro tip

Dispatch is especially powerful when you need Claude to interact with GUI applications that don't have APIs or CLI tools. Anything that requires clicking, Dispatch handles.

Chrome Extension

Let Claude taste its own cooking.

The analogy: Would you trust a chef who can't taste their own food? The Chrome extension gives Claude Code eyes. It can visually inspect what it built, check if it matches your requirements, fix issues, and loop until it looks right.

Without this, Claude writes code and hopes for the best. With this, it writes code, checks the browser, sees the result, fixes issues, and iterates. Massive difference in output quality.

The recursive loop:

- Claude Code writes the code
- Checks it in the browser via Claude in Chrome
- Sees any visual issues
- Fixes them
- Checks again
- Repeats until it matches your specs

Make it default with /config:

Run /config to see all your settings. Scroll to the bottom and you'll find:

- **Claude in Chrome enabled by default** - Set to true so it always uses the browser when relevant
- **Enable control for all sessions** - Set to true so every session can use browser tools

```
/config
```

Pro tip

If you have multiple browser tools (Claude in Chrome, Agent Browser, etc.) and there's overlap, specify which one to use: "use Claude in Chrome for this task." This avoids confusion about which browser tool fires. Claude in Chrome is best for pre-authenticated apps since it uses your actual browser, no re-login needed.

08

Live Preview

See your web app update in real time.

This one works in the **Claude Desktop app's chat mode**, not the Claude Code terminal. When you ask Claude to build a website or HTML file, instead of just giving you a file to download, you can click the sidebar and get a live preview that updates as Claude makes changes.

If you want the path of least resistance for drafting a quick site, landing page, or prototype without spinning up a local server, this is it.

When to use this vs Claude Code

Live Preview is great for quick drafts and visual prototyping. For real development with git, testing, and deployment, use Claude Code in your terminal with the Chrome extension for validation.

09

Session Forking

Branch your conversation to explore alternatives.

Sometimes you hit a fork in the road. Path A seems right, but you want to explore Path B without risking your progress. `/branch` creates a copy of your conversation at that exact point. You can take the branch in a completely different direction while your original stays untouched.

Real example: You're refining a client proposal. The current version looks good. But you want to experiment with very aggressive pricing without flooding the context window and confusing Claude about which pricing is real.

```
/branch very aggressive pricing version
```

Now you're in a separate branch where you can experiment freely. If this version wins, keep going. If not, resume the original:

```
claude -r <session-id>
```

Pro tip

Use `/branch` whenever you want to experiment without commitment. The original conversation stays frozen exactly where you left it.

10

/btw Side Queries

Ask a quick question without interrupting.

/btw lets you multitask. While Claude is executing a long task, you can fire off a quick question that doesn't flood the main context window. Claude answers your side question and keeps working on the primary task.

```
/btw what are our numbers for Q1?
```

Hit space after the answer to dismiss it and the main conversation keeps flowing.

Pro tip

Use /btw for quick sanity checks, looking up numbers, or asking clarifying questions while Claude is in the middle of something big. It won't derail the main work.

11

Git Worktrees

Three desks. Three agents. Zero overlap.

The analogy: Imagine three separate desks in your office. Each desk has its own agent, its own documents, and its own task. Desk one is writing a client proposal. Desk two is redesigning the website. Desk three is building the quarterly report. None of them can touch each other's files.

That's what worktrees give you. Each one is an isolated copy of your project where Claude Code can work without stepping on the other sessions' files.

The syntax:

```
claude -w proposal-acme
```

The `-w` flag stands for worktree. Everything after it is the name. Then just prompt normally:

```
claude -w proposal-acme
Write a revised proposal for Acme Corp based on their project notes in
/clients/acme. They're now in phase 2. Propose an upsell to enterprise
tier for phase 3.
```

You can run as many as you want simultaneously:

```
claude -w website-update
Add testimonials to the bottom of our website using real client names
from /clients
```

```
claude -w quarterly-report
Read reports from /reports and create a professional Q1 board report
```

Pro tip

If you've been intimidated by worktrees, don't be. It's literally just `-w` plus a name. Each one gets its own isolated lane so they can't conflict.

12

Batch Command

Fan out across dozens of agents at once.

Take this one with a grain of salt. If worktrees let you run 3 agents, batch lets you run 5, 10, 30, or more. It prioritizes breadth of execution over depth and thoughtfulness of each individual agent.

How it works:

- You give `/batch` a request that applies to many files
- Claude enters **Plan Mode** first, researching the scope
- It creates a **worker instructions template** with exact changes per file
- Then it spins up as many agents as needed to execute in parallel

```
/batch update the hourly rate from 150 to 175 across all proposal files
in /proposals. Keep existing formatting. Recalculate all line items
totals and the grand total in each proposal.
```

Token warning

Batch will employ as many agents as it wants. If you are rich in tokens, go for it. If you're being frugal or watching your rate limits, avoid this for now. Each worker consumes tokens independently.

13

The --bare Flag

Strip everything. Run fast.

When you start Claude Code normally, it loads your CLAUDE.md, skills, hooks, MCP servers, and everything else in your .claude folder. The --bare flag skips all of that. Result: startup is up to 10x faster.

This is essential for automation, scripts, and CI pipelines where you don't need the full configuration loaded.

```
claude --bare -p 'your prompt here'
```

Important caveat:

If you're using a Max subscription (not an API key), --bare may not work. It appears to require API authentication. The -p flag alone (without --bare) does work with your Max account and still gives you non-interactive execution, just without the speed boost of skipping auto-discovery.

The -p flag on its own:

```
claude -p 'what is 2+2'
```

This runs Claude Code non-interactively, gets the answer, and exits. Useful for quick one-shot questions from the terminal without opening a full session.

Pro tip

Use --bare + -p together for automation scripts. Use -p alone for quick questions. Use neither for normal interactive sessions where you want all your rules and skills loaded.

The --add-dir Flag

Give Claude access to multiple projects.

By default, Claude Code only has access to the folder you opened it in. The `--add-dir` flag lets you bring additional folders into the session so Claude can read, write, and navigate across multiple repos at once.

```
claude --add-dir templates --add-dir brand-assets
```

Now Claude can pull from your templates folder AND your brand assets folder, even though they're outside your current project. Both directories are added to the session with full access.

Pro tip

Perfect for monorepo-adjacent workflows. If your frontend and backend live in different repos, add both and Claude can coordinate changes across them in a single session.

Custom Agents

Start a session with a specialized persona.

Instead of starting a Claude Code session and then explaining who it should be, you can create agents that come pre-loaded with their own personality, rules, and tool access. Think of them as specialized team members you can summon by name.

Where agents live:

```
.claude/agents/proposal-writer.md
```

The filename IS the agent name. Inside the markdown file, you define:

- **Name** and **description** of what this agent does
- **Tools** this agent can invoke
- **Rules** and behavior instructions
- The structure of how it should execute tasks

Start a session with your agent:

```
claude --agent proposal-writer
```

Now when you say 'write a proposal for Tech Corp,' it uses all the rules, the personality, the tool access, and the structure you defined. Every time.

Pro tip

Start with one agent for your most common workflow. A proposal writer, a code reviewer, a documentation agent. Once you see how much time it saves, you'll want to create agents for everything.

16

Bonus: Voice Input

Code by talking, not typing.

Boris says he does most of his coding by speaking to Claude rather than typing. The `/voice` command lets you dictate prompts naturally. For complex instructions, describing what you want verbally is often faster and more expressive than typing it out.

```
/voice
```

Pro tip

Voice is particularly powerful for explaining architecture decisions, describing bugs, or giving multi-step instructions. Talk like you're explaining to a colleague sitting next to you.

Cheat Sheet

Every command from the video, copy-paste ready.

Teleport to desktop

Resume a mobile session on your computer.

```
claude --teleport
```

Remote control

Control a desktop session from your phone.

```
/remote-control
```

Name a remote session

Label it so you know which is which.

```
/remote-control "website redesign"
```

Loop a task

Run a command every N minutes (terminal must stay open).

```
/loop 30m check my inbox and draft replies
```

Schedule in the cloud

Long-running recurring tasks that survive restarts.

```
/schedule
```

Set up a hook

Tag the guide agent for correct syntax.

```
set up a hook so that every time I start a session, you read the brand guidelines @claude-code-guide
```

View/edit all settings

See every toggle including Chrome and remote control.

```
/config
```

Fork a session

Branch the conversation to explore alternatives.

```
/branch aggressive pricing version
```

Resume original session

Get back to where you branched from.

```
claude -r <session-id>
```

Side query

Ask without interrupting the main task.

```
/btw what are our Q1 numbers?
```

Start a worktree

Isolated parallel session with its own files.

```
claude -w proposal-acme
```

Batch across files

Fan out changes across many files at once.

```
/batch update the rate from 150 to 175 across all proposals
```

Fast non-interactive run

Skip auto-discovery, run a prompt, exit.

```
claude --bare -p 'your prompt'
```

Quick one-shot question

Run a prompt without opening a full session.

```
claude -p 'what is 2+2'
```

Add another directory

Give Claude access to folders outside your project.

```
claude --add-dir templates --add-dir brand-assets
```

Start with a custom agent

Load a specialized persona by name.

```
claude --agent proposal-writer
```

Voice input

Dictate instead of typing.

```
/voice
```

Want to go deeper?

If you want the full Claude Code system, live walkthroughs of features like these as they drop, and direct access to the community building with these tools every day, check out the Early AI Dopters community.

Join the community: Early AI Dopters
<https://www.skool.com/earlyaidopters/about>